

Binary Search — Master Revision Sheet (Step 4, P01-P32)

One-stop revision. Each entry = **Problem** (what's asked + tiny example) + **Recall** (the trigger line) + **Algorithm** + **Trap / don't-miss** + **Time / Space** + **Pattern**. Go top-to-bottom the night before an interview; every Binary Search problem in this step is here.

60-Second Index

#	Problem	LC	Pattern	Time / Space
P01	Binary Search	704	Classic BS	$O(\log N) / O(1)$
P02	Lower Bound	GFG	First $\geq x$	$O(\log N) / O(1)$
P03	Upper Bound	GFG	First $> x$	$O(\log N) / O(1)$
P04	Search Insert Position	35	Lower bound	$O(\log N) / O(1)$
P05	Floor & Ceil	GFG	Boundary search	$O(\log N) / O(1)$
P06	First & Last Position	34	Two biased BS	$O(\log N) / O(1)$
P07	Count Occurrences	GFG	last – first + 1	$O(\log N) / O(1)$
P08	Rotated Array Search I	33	Sorted-half check	$O(\log N) / O(1)$
P09	Rotated Array Search II	81	Sorted-half + dup trim	$O(\log N) \sim O(N) / O(1)$
P10	Min in Rotated Array	153	Eliminate sorted half	$O(\log N) / O(1)$
P11	Count Rotations	GFG	Index of min	$O(\log N) / O(1)$
P12	Single Element	540	Index parity	$O(\log N) / O(1)$
P13	Peak Element	162	BS on slope	$O(\log N) / O(1)$
P14	Square Root	69	BS on answer	$O(\log x) / O(1)$
P15	Nth Root	GFG	BS on answer + power	$O(n \cdot \log m) / O(1)$
P16	Koko Eating Bananas	875	BS on answer (min rate)	$O(n \cdot \log \max) / O(1)$
P17	Min Days M Bouquets	1482	BS on answer + greedy	$O(n \cdot \log \max \text{Day}) / O(1)$
P18	Smallest Divisor	1283	BS on answer (ceil-sum)	$O(n \cdot \log \max) / O(1)$
P19	Ship Capacity in D Days	1011	BS on answer + greedy	$O(n \cdot \log \text{sum}) / O(1)$
P20	Kth Missing Positive	1539	BS on index (missing)	$O(\log N) / O(1)$
P21	Aggressive Cows	SPOJ	BS on answer (maximize min)	$O(n \log n) / O(1)$
P22	Book Allocation	GFG	BS on answer (min max)	$O(n \cdot \log \text{sum}) / O(1)$
P23	Split Array Largest Sum	410	= Book Allocation	$O(n \cdot \log \text{sum}) / O(1)$
P24	Painter's Partition	GFG	= Book Allocation	$O(n \cdot \log \text{sum}) / O(1)$
P25	Min Max Gas Station Dist	GFG	Real-valued BS	$O(n \cdot \log(\text{range}/\epsilon)) / O(1)$
P26	Median of Two Sorted Arrays	4	Partition BS	$O(\log \min(m, n)) / O(1)$
P27	Kth Element of Two Arrays	GFG	Partition BS (left=k)	$O(\log \min(m, n)) / O(1)$
P28	Row with Max 1s	GFG	Lower bound per row	$O(n \cdot \log m) / O(1)$
P29	Search 2D Matrix I	74	Flatten to 1D BS	$O(\log(n \cdot m)) / O(1)$
P30	Search 2D Matrix II	240	Staircase from corner	$O(n+m) / O(1)$
P31	Peak Element 2D	1901	BS on columns (col max)	$O(n \cdot \log m) / O(1)$
P32	Matrix Median	GFG	BS on value + count $\leq$ mid	$O(\log V \cdot n \cdot \log m) / O(1)$

Pattern Toolbox (the shapes that repeat)

- **Overflow-safe mid:** always `mid = low + (high-low)/2`, never `(low+high)/2` for large indices.
- **Exact search:** loop `low <= high`, return on match, else move `low=mid+1` / `high=mid-1`. (P01, P08, P09, P29, P30)
- **Boundary search (lower/upper bound):** store a candidate on the "good" side and keep shrinking. Lower = first `>= x`, Upper = first `> x`. (P02-P07, P28)
- **Eliminate the sorted half:** in a rotated array, one half is always sorted — check the target/min against it and discard. (P08-P11)
- **Binary search on the answer:** the answer is a *value*, not an index. Search `[minAns, maxAns]`, test a **monotonic feasibility predicate**, minimize/maximize. (P14-P25, P32)
- *Minimize feasible* (Koko, divisor, ship, book/split/painter): feasible $\rightarrow$ `high=mid-1`, keep candidate.
- *Maximize feasible* (Aggressive Cows): feasible $\rightarrow$ `low=mid+1`, keep candidate.
- *Real-valued* (gas station): loop while `high-low > 1e-6`, `low=mid` / `high=mid` (no ± 1).
- **Partition two arrays:** cut the smaller array; a valid partition satisfies `l1<=r2 && l2<=r1`; use `INT_MIN` / `INT_MAX` sentinels. (P26, P27)
- **2D tricks:** globally sorted $\rightarrow$ flatten (`mid/m`, `mid%m`); row+col sorted $\rightarrow$ staircase from top-right; peak $\rightarrow$ BS on columns; median $\rightarrow$ BS on value + per-row count.

The "binary search on answer" checklist: (1) Is the answer monotonic? (2) What are `low` / `high` bounds? (3) What's the $O(n)$ feasibility check? (4) Minimize or maximize? Answer those four and the code writes itself.

P01 — Binary Search

- **Problem:** Find `target`'s index in a sorted array; `-1` if absent. `[-1,0,3,5,9,12]`, `t=9` $\rightarrow$ 4.
- **Recall:** "Compare with mid; keep the half that can contain target."
- **Algo:** `low<=hi`; `==` return; `<target` $\rightarrow$ `low=mid+1`; else `high=mid-1`. Recursive or iterative.
- **Trap:** Recurse/move with the correct bound (`low`, not `0`); never `low=mid` / `high=mid` (infinite loop). Sorted input only.
- **$O(\log N)$ / $O(1)$** (recursion $O(\log N)$ stack).
- **Pattern:** Classic binary search.

P02 — Lower Bound

- **Problem:** First index with `arr[i] >= x` (else `n`). `[3,5,8,15,19]`, `x=9` $\rightarrow$ 3.
- **Recall:** "First `>= x`. On `arr[mid]>=x` store & go left."
- **Algo:** `ans=n`; `arr[mid]>=x` $\rightarrow$ `ans=mid`, `high=mid-1`; else `low=mid+1`.
- **Trap:** `>=` (not `>`, that's upper bound). Default `ans=n` for "all smaller".
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** Boundary search. STL `lower_bound`.

P03 — Upper Bound

- **Problem:** First index with `arr[i] > x` (else `n`). `[3,5,8,9,15,19]`, `x=9` $\rightarrow$ 4.
- **Recall:** "First strictly `> x`. On `arr[mid]>x` store & go left."
- **Algo:** `ans=n`; `arr[mid]>x` $\rightarrow$ `ans=mid`, `high=mid-1`; else `low=mid+1`.
- **Trap:** `>` vs `>=` is the *only* difference from lower bound. `count(x) = upper - lower`.
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** Boundary search. STL `upper_bound`.

P04 — Search Insert Position

- **Problem:** Index where `x` is, or where it'd be inserted. `[1,2,4,7]`, `x=6` $\rightarrow$ 3.
- **Recall:** "Insert position = lower bound of `x`."
- **Algo:** Exactly lower bound.
- **Trap:** Return an **insert index**, never `-1`. Default `n` for "past the end".
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** Lower bound reframed.

P05 — Floor & Ceil in Sorted Array

- **Problem:** Floor = largest `<= x`; Ceil = smallest `>= x` (values; `-1` if none). `[3,4,4,7,8,10]`, `x=5` $\rightarrow$ floor 4, ceil 7.

- **Recall:** "Floor: $\leq x$ → store & go right. Ceil: $\geq x$ → store & go left."
- **Algo:** Two BS; store `arr[mid]` (the value).
- **Trap:** Store the **value** not the index; floor goes right on hit, ceil left; default `-1`. Ceil = lower-bound value.
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** Boundary search (value form).

P06 — First & Last Position

- **Problem:** First and last index of `target` (dups). `[5,7,7,8,8,10]`, `t=8` → `[3,4]`; absent → `[-1,-1]`.
- **Recall:** "First: match → go left. Last: match → go right."
- **Algo:** Two biased BS; on `==` store and move (first `high=mid-1`, last `low=mid+1`).
- **Trap:** Don't return on first match. `first==-1` → `[-1,-1]`. (Bound view: `first=lower`, `last=upper-1`.)
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** Two biased binary searches.

P07 — Count Occurrences

- **Problem:** Count of `x` in sorted array. `[2,4,6,8,8,8,11,13]`, `x=8` → 3.
- **Recall:** "count = last – first + 1."
- **Algo:** first & last (P06); `count = last-first+1`; 0 if absent.
- **Trap:** Don't forget `+1`; short-circuit to 0 when `first==-1`.
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** first/last then subtract (`upper – lower`).

P08 — Search in Rotated Sorted Array I

- **Problem:** Search in a rotated array of **distinct** elements. `[4,5,6,7,0,1,2]`, `t=0` → 4.
- **Recall:** "One half is always sorted — check if target is in it."
- **Algo:** `==` → return. `nums[low]<=nums[mid]` (left sorted): `low<=t<mid` → left else right. Else right sorted: `mid<t<=high` → right else left.
- **Trap:** `<=` in `nums[low]<=nums[mid]` (handles no-rotation). Range boundaries `<` vs `<=`.
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** Sorted-half elimination.

P09 — Search in Rotated Sorted Array II

- **Problem:** Same but **duplicates allowed**; return bool. `[2,5,6,0,0,1,2]`, `t=0` → true.
- **Recall:** "P08 + guard: `nums[low]==nums[mid]==nums[high]` → `low++,high--`."
- **Algo:** Add the endpoint-equal trim (with `continue`); rest identical to P08.
- **Trap:** Need all three equal before trimming; `continue` after; returns bool not index.
- **$O(\log N)$ avg, $O(N)$ worst / $O(1)$.**
- **Pattern:** Sorted-half + duplicate trim.

P10 — Find Minimum in Rotated Sorted Array

- **Problem:** Minimum of a rotated distinct array. `[4,5,6,7,0,1,2]` → 0.
- **Recall:** "Min is in the unsorted half; grab the sorted half's start."
- **Algo:** Early exit if `nums[low]<nums[high]`. Left sorted → candidate `nums[low]`, go right; else candidate `nums[mid]`, go left. (Compact: compare `nums[mid]` vs `nums[high]`, `low<high`, return `nums[low]`.)
- **Trap:** Prefer comparing with `nums[high]`; `low=mid` → infinite loop in compact form.
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** Sorted-half elimination.

P11 — Count Rotations

- **Problem:** How many times rotated. `[4,5,6,7,0,1,2,3]` → 4.
- **Recall:** "Rotations = index of minimum."
- **Algo:** P10 compact, return the **index** `low`.
- **Trap:** Return index (`low`), not value; `high=mid` (not `mid-1`), loop `low<high`.

- **$O(\log N)$ / $O(1)$.**
- **Pattern:** Index of minimum.

P12 — Single Element in Sorted Array

- **Problem:** All twice except one; find it ($O(\log N)$). `[1,1,2,3,3,4,4,8,8]` → 2.
- **Recall:** "Pairs are even-indexed before the single; make mid even."
- **Algo:** Force `mid` even; `nums[mid]==nums[mid+1]` → `low=mid+2` (single right); else `high=mid`.
- **Trap:** Must normalize mid to even; `mid+2` (skip full pair); `high=mid` (candidate stays).
- **$O(\log N)$ / $O(1)$.** (XOR is $O(N)$ — doesn't qualify.)
- **Pattern:** Index parity.

P13 — Find Peak Element

- **Problem:** Any index strictly greater than neighbours (`nums[-1]=nums[n]=-∞`). `[1,2,3,1]` → 2.
- **Recall:** "Walk uphill: `nums[mid]<nums[mid+1]` → go right."
- **Algo:** `low<high`; ascending → `low=mid+1`; else `high=mid`; return `low`.
- **Trap:** `low<high` (not `<=`); `high=mid`; compare `mid` vs `mid+1` only.
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** BS on slope.

P14 — Square Root (Floor)

- **Problem:** `[√x]` without `sqrt`. `x=8` → 2.
- **Recall:** "Largest mid with `mid*mid <= x`."
- **Algo:** BS `[1, x/2]`; valid → `ans=mid, left=mid+1`; else `right=mid-1`.
- **Trap:** `long long` for `mid*mid`; base case `x<2`; return stored `ans`.
- **$O(\log x)$ / $O(1)$.**
- **Pattern:** BS on the answer.

P15 — Nth Root

- **Problem:** Integer `x` with `x^n = m`, else `-1`. `n=3, m=27` → 3.
- **Recall:** "BS `[1, m]`; compute `mid^n` with early overflow break."
- **Algo:** Power loop breaking when `>m`; `==m` → return, `<m` → right, `>m` → left.
- **Trap:** `long long` product + early break; exact `==m` check (else return `-1`).
- **$O(n \cdot \log m)$ / $O(1)$.**
- **Pattern:** BS on the answer + guarded power.

P16 — Koko Eating Bananas

- **Problem:** Min integer speed to eat all piles within `h` hours. `[3,6,7,11]`, `h=8` → 4.
- **Recall:** "BS on speed `[1, max(pile)]`; `hours = Σ ceil(pile/speed)`."
- **Algo:** Feasible (`hours <= h`) → `high=mid-1`, keep candidate; else `low=mid+1`.
- **Trap:** `ceil = (pile+speed-1)/speed`; sum in `long long`; `high=max(pile)`.
- **$O(n \cdot \log \max)$ / $O(1)$.**
- **Pattern:** BS on answer (minimize feasible).

P17 — Minimum Days for M Bouquets

- **Problem:** Min day to make `m` bouquets of `k adjacent` flowers. `[1,10,3,10,2]`, `m=3, k=1` → 3.
- **Recall:** "BS on day; greedily count adjacent bloomed flowers."
- **Algo:** `m*k > n` → `-1`. `canMake` counts runs of `k`, reset on gap. Feasible → minimize.
- **Trap:** Reset flower run on a non-bloomed flower; `m*k > n` check (cast `long long`).
- **$O(n \cdot \log \maxDay)$ / $O(1)$.**
- **Pattern:** BS on answer + greedy check.

P18 — Smallest Divisor Given a Threshold

- **Problem:** Smallest `d` with $\sum \text{ceil}(\text{num}/d) \leq \text{threshold}$. `[1,2,5,9]`, `thr=6` → 5.
- **Recall:** "BS on divisor `[1,max]` ; sum of ceils decreases as `d` grows."
- **Algo:** Feasible → `high=mid-1` , keep; else `low=mid+1` .
- **Trap:** Integer ceil; sum decreases with `d` (don't flip direction); `high=max(nums)` .
- **$O(n \cdot \log \max)$ / $O(1)$.**
- **Pattern:** BS on answer (twin of Koko).

P19 — Capacity to Ship Packages in D Days

- **Problem:** Min ship capacity to ship ordered weights in `days` . `[1..10]` , `d=5` → 15.
- **Recall:** "BS on capacity `[max(w), sum(w)]` ; greedy day count."
- **Algo:** `canShip` starts a new day on overflow (`load=w`). `days<=D` → minimize.
- **Trap:** `low=max(w)` (heaviest must fit); reset `load=w` not `0` ; overflow in sum.
- **$O(n \cdot \log \text{sum})$ / $O(1)$.**
- **Pattern:** BS on answer + greedy partition.

P20 — Kth Missing Positive Number

- **Problem:** Kth missing positive in a strictly increasing array. `[2,3,4,7,11]` , `k=5` → 9.
- **Recall:** " `missing = arr[mid] - (mid+1)` ; find first index with `missing>=k` ; answer `low+k` ."
- **Algo:** `missing<k` → `low=mid+1` ; else `high=mid-1` ; return `low+k` .
- **Trap:** Off-by-one in `arr[i] - (i+1)` ; answer is `low+k` (BS on index, not value).
- **$O(\log N)$ / $O(1)$.**
- **Pattern:** BS on index via missing-count.

P21 — Aggressive Cows

- **Problem:** Place `cows` to **maximize the minimum** gap. `[1,2,8,4,9]` , `cows=3` → 3.
- **Recall:** "Sort. BS on gap; greedy place cows $\geq d$ apart. Maximize."
- **Algo:** Feasible → `ans=mid` , `low=mid+1` (maximize); else `high=mid-1` .
- **Trap:** **Sort first**; maximize direction (feasible → go right); track last *placed* cow.
- **$O(n \log n + n \cdot \log(\text{range}))$ / $O(1)$.**
- **Pattern:** BS on answer (maximize minimum).

P22 — Book Allocation

- **Problem:** `m` students, contiguous books, minimize the max pages. `[25,46,28,49,24]` , `m=4` → 71.
- **Recall:** "BS on page limit `[max, sum]` ; `students>m` → go right."
- **Algo:** Greedy `countStudents` ; `>m` → `low=mid+1` , else `high=mid-1` ; return `low` .
- **Trap:** `low=max(arr)` ; `m>n` → -1; contiguous only; answer converges to `low` .
- **$O(n \cdot \log \text{sum})$ / $O(1)$.**
- **Pattern:** BS on answer (minimize maximum).

P23 — Split Array Largest Sum

- **Problem:** Split into `k` contiguous parts, minimize the largest part sum. `[7,2,5,10,8]` , `k=2` → 18.
- **Recall:** "Same as Book Allocation: BS on max-sum, greedy split count."
- **Algo:** `splitArr<=k` → `high=mid-1` , keep; else `low=mid+1` .
- **Trap:** `low=max(nums)` ; `<=k` (fewer splits fine); handle `k>n` .
- **$O(n \cdot \log \text{sum})$ / $O(1)$.**
- **Pattern:** BS on answer (minimize maximum).

P24 — Painter's Partition

- **Problem:** `k` painters, contiguous boards, minimize the max time. `[10,20,30,40]` , `k=2` → 60.
- **Recall:** "Identical to Book Allocation / Split Array."
- **Algo:** Greedy `countPainters` ; `>k` → go right, else record & go left.

- **Trap:** Contiguous blocks; `low=max(boards)`; `>k` vs `>=k` direction.
- **$O(n \cdot \log \text{sum}) / O(1)$.**
- **Pattern:** BS on answer (minimize maximum).

P25 — Minimize Max Distance Between Gas Stations

- **Problem:** Add `k` stations to minimize the max adjacent gap (**real** answer). `[1,2,3,4,5]`, `k=4` → 0.5.
- **Recall:** "Real-valued BS on gap; stations in gap = `floor(g/dist)` (-1 if exact)."
- **Algo:** `while(high-low>1e-6)`; `cnt>k` → `low=mid`; else `high=mid`; return `high`.
- **Trap:** No `±1` (continuous); exact-divisibility -1 fix; `high=max initial gap`.
- **$O(n \cdot \log(\text{range}/\epsilon)) / O(1)$.** (Heap alt: $O(n \log n + k \log n)$.)
- **Pattern:** Real-valued BS on the answer.

P26 — Median of Two Sorted Arrays

- **Problem:** Median of two sorted arrays in $O(\log(\min))$. `[1,2]`, `[3,4]` → 2.5.
- **Recall:** "Partition smaller array; `left=(m+n+1)/2`; valid `l1<=r2 && l2<=r1`."
- **Algo:** Odd → `max(l1,l2)`; even → `(max(l1,l2)+min(r1,r2))/2`. `l1>r2` → `high=mid1-1` else `low=mid1+1`.
- **Trap:** Search **smaller** array; `INT_MIN/INT_MAX` sentinels; `(m+n+1)/2`; double-cast the even average.
- **$O(\log \min(m,n)) / O(1)$.**
- **Pattern:** Partition binary search.

P27 — Kth Element of Two Sorted Arrays

- **Problem:** Kth smallest across two sorted arrays. `[2,3,6,7,9]`, `[1,4,8,10]`, `k=5` → 6.
- **Recall:** "Partition, left half size = `k`; cut range `[max(0,k-n), min(k,m)]`."
- **Algo:** Valid `l1<=r2 && l2<=r1` → `max(l1,l2)`.
- **Trap:** Clamp cut bounds; search smaller array; sentinels. (Median = this with `k=(m+n+1)/2`.)
- **$O(\log \min(m,n)) / O(1)$.**
- **Pattern:** Partition binary search.

P28 — Row with Maximum 1s

- **Problem:** Row (sorted 0s→1s) with most 1s; earliest on tie. `[[1,1,1],[0,0,1],[0,0,0]]` → 0.
- **Recall:** "ones = `m - lowerBound(1)`; max over rows."
- **Algo:** Per-row lower bound of 1; track max with strict `>`.
- **Trap:** Rows must be sorted; strict `>` keeps earliest tie; `ones = m - firstOneIdx`.
- **$O(n \cdot \log m) / O(1)$.**
- **Pattern:** Lower bound per row.

P29 — Search in 2D Matrix I

- **Problem:** Globally sorted matrix (row-start > prev row-end); find target. `target=8` → true.
- **Recall:** "Flatten to 1D; `matrix[mid/m][mid%m]`."
- **Algo:** BS `[0, n*m-1]`; standard compare.
- **Trap:** Divide/mod by `m` (columns); `high=n*m-1`; only for globally-sorted.
- **$O(\log(n \cdot m)) / O(1)$.**
- **Pattern:** Flatten to 1D.

P30 — Search in 2D Matrix II

- **Problem:** Rows sorted **and** columns sorted, not globally; find target. `target=8` → true.
- **Recall:** "Top-right start; `>target` → left, `<target` → down."
- **Algo:** From `(0,m-1)`: `==` → true, `<t` → `row++`, `>t` → `col--`.
- **Trap:** Start at a **corner** (top-right/bottom-left), not top-left; can't flatten here.
- **$O(n+m) / O(1)$.**
- **Pattern:** Staircase search.

P31 — Find Peak Element in 2D Grid

- **Problem:** Any cell strictly greater than its 4 neighbours; return `[r,c]`.
- **Recall:** "BS on columns; take the column's max row; move to bigger horizontal side."
- **Algo:** Col max dominates vertically; `>` both L/R $\rightarrow$ peak; `left>mid` $\rightarrow$ `high=mid-1`; else `low=mid+1`.
- **Trap:** Must use the column **max** (else vertical neighbours not guaranteed smaller); check both L & R; `-∞` at borders.
- **$O(n \cdot \log m)$ / $O(1)$.**
- **Pattern:** BS on columns.

P32 — Median of Row-wise Sorted Matrix

- **Problem:** Median of a row-sorted matrix (odd total). `[[1,3,5],[2,6,9],[3,6,9]]` $\rightarrow$ 5.
- **Recall:** "BS on value; median = smallest x with `count( $\leq$ x)  $\geq$  (n*m+1)/2`."
- **Algo:** `count =  $\sum$  upper_bound(row, mid)`; `count < half` $\rightarrow$ `low=mid+1`; else `high=mid`; return `low`.
- **Trap:** Count `$\leq$  mid` (`upper_bound`, not `lower_bound`); half = `(n*m+1)/2`; `low<high`, `high=mid`.
- **$O(\log(\text{maxVal}) \cdot n \cdot \log m)$ / $O(1)$.**
- **Pattern:** BS on value + per-row count.

Decision Guide — "Which binary search is this?"

Clue in the problem	Go to
"sorted array, find target"	P01 classic
"first $\geq$ / first $>$ / insert position / floor / ceil"	P02-P05 bounds
"first & last / count of x"	P06-P07 biased BS
"rotated sorted array"	P08-P11 sorted-half
"every element twice except one"	P12 parity
"peak / greater than neighbours"	P13 (1D), P31 (2D)
"min/max value so that a condition holds"	P14-P25 BS on answer
"minimize the maximum / maximize the minimum"	P21-P24 partition-style
"median / Kth across two sorted arrays"	P26-P27 partition
"search / peak / median in a matrix"	P28-P32 2D

Golden Rules

1. **Overflow-safe mid** always: `low + (high-low)/2`.
2. **Exact match** $\rightarrow$ `low<=high`; **converge to a boundary** $\rightarrow$ `low<high` with `high=mid`.
3. **Move at least one pointer past mid** (`mid $\pm$ 1`) unless you deliberately keep `mid` as a candidate with `high=mid`.
4. **BS on answer:** confirm monotonicity, set `[low,high]`, write the $O(n)$ feasibility check, decide minimize vs maximize.
5. **Two-array partition:** always cut the **smaller** array; guard with `INT_MIN` / `INT_MAX`.
6. **Real-valued BS:** loop on precision (`> 1e-6`), assign `low=mid` / `high=mid` (no ± 1).